선물 모의거래의 흐름과 구현

분기·복구·저장을 구분해 읽는 개발 가이드

주문은 준비 상태를 확인하고,
청산은 감시와 검증이 처리합니다.

01 거래 흐름

전체 구조부터 충전·주문·종료·청산·복구까지
판단은 마름모, 분기는 이름 있는 화살표로 표현

02 구현 해설

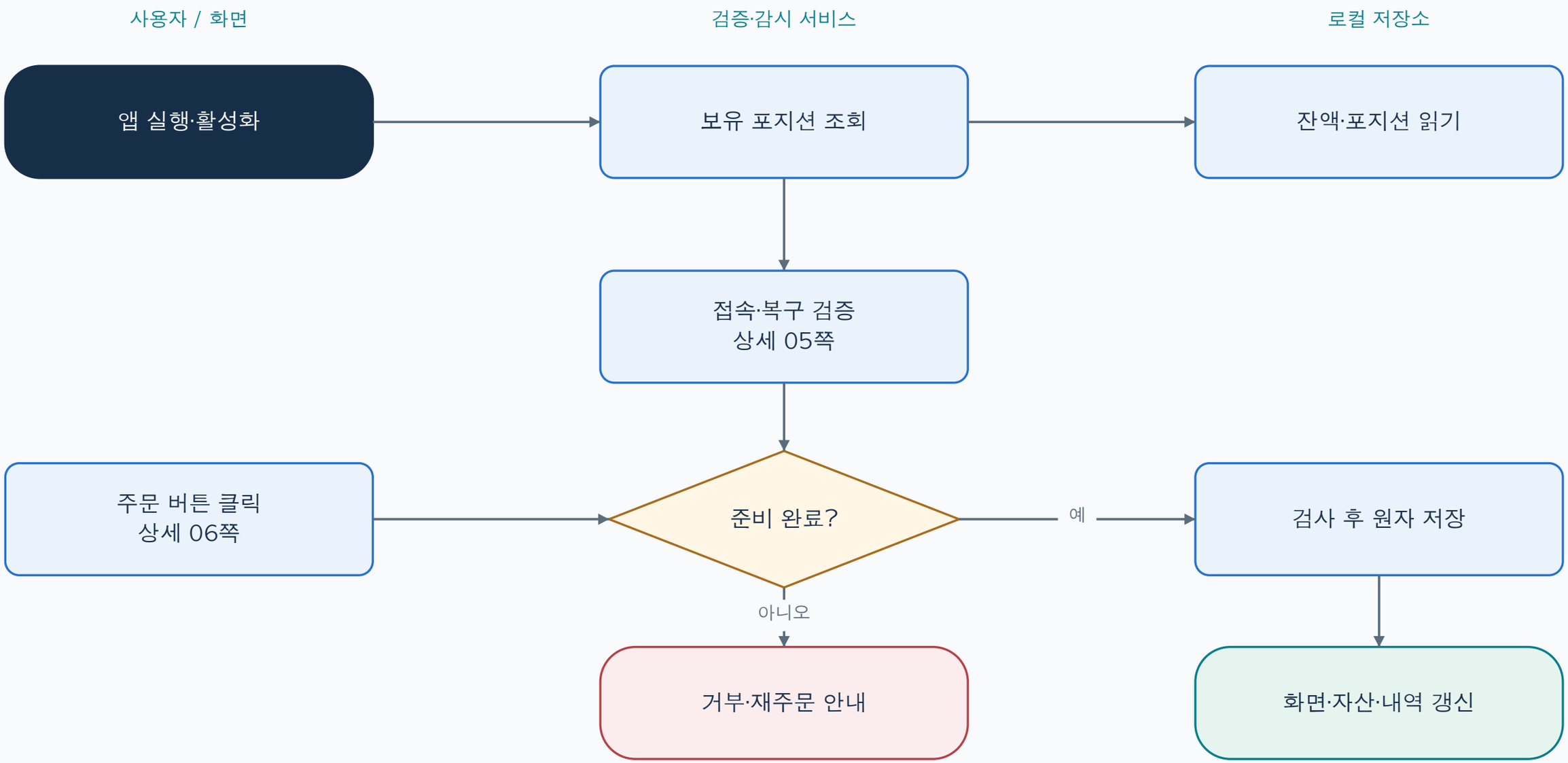
Futures 파일의 책임과 핵심 메서드
계산·진행·저장·화면의 경계를 나누어 설명

읽는 방법

파랑 사각형 = 처리 황색 마름모 = 판단 초록 = 완료 빨강 = 거부·보류
재시도는 되돌아가는 화살표로, 다른 상세 흐름으로 이동하는 경우는 페이지 번호로 표시합니다.

01 전체 구조: 접속, 주문, 감시

주문 경로와 청산 경로는 저장소에서 만납니다. 실시간 감시와 과거 검증은 독립적으로 진행됩니다.



별도 감시: 소켓 가격은 즉시 청산 판정(08쪽), 과거 검증은 누락 구간 판정(09쪽)을 수행합니다. 주문 자체는 과거 API를 호출하지 않습니다.

02 거래 정책과 데이터 범위

이 문서의 차트는 현재 국내 모의 선물 정책을 기준으로 합니다.

계좌·충전

업비트·빗썸의 현물·선물 계좌는 독립됩니다. 선물은 KRW 마켓만 지원하고 이체는 없습니다.
신규 선물 계좌는 0원, 광고 보상은 해당 계좌에 1,000만 원입니다. 최초 100만 원은 업비트 현물에만 지급합니다.

주문·포지션

시장가 방식으로 클릭 당시 표시 소켓 가격을 고정합니다. 동일 종목은 활성 포지션 하나이며 같은 방향·레버리지만 추가할 수 있습니다.
수동 종료는 전량 종료이고 부분 종료는 없습니다.

레버리지

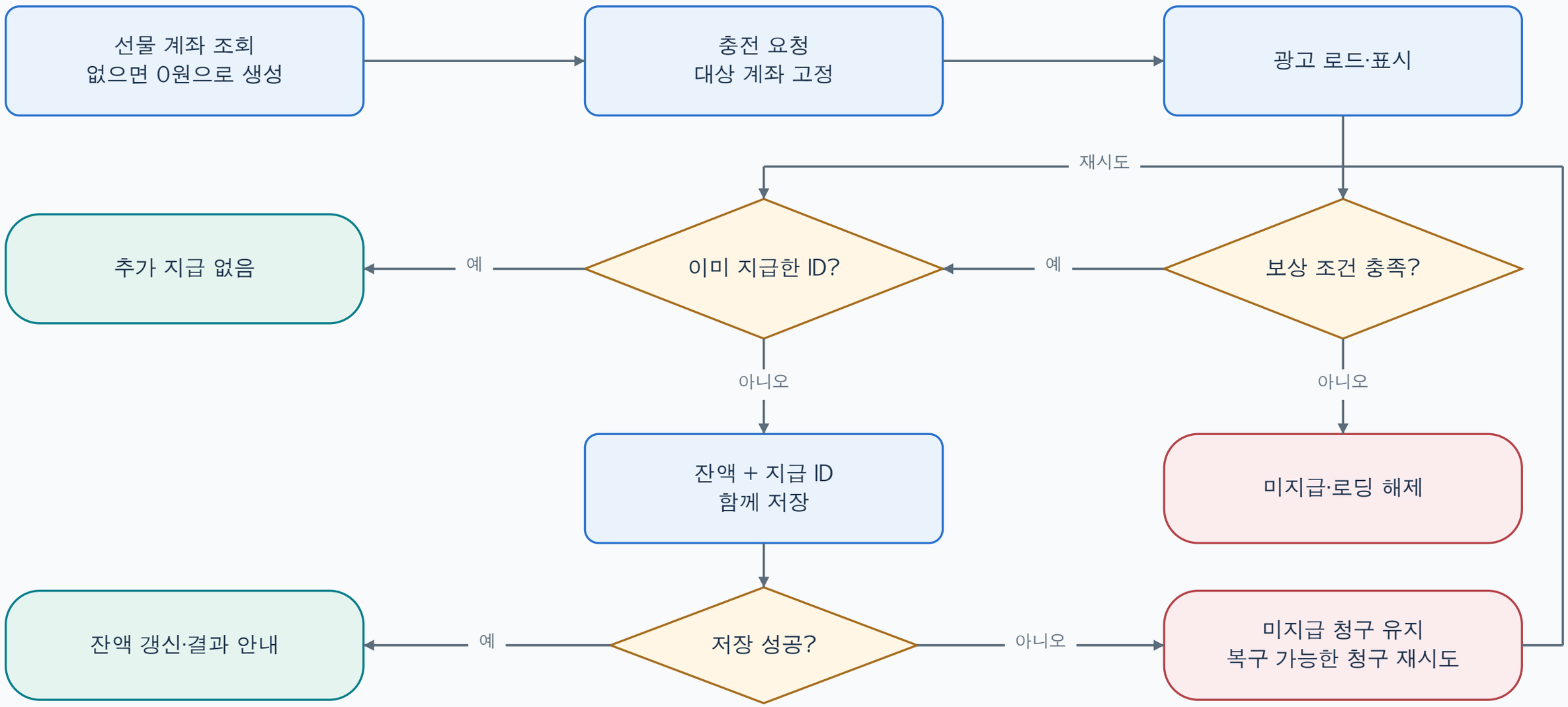
BTC·ETH: 최대 100배
SOL, XRP, ADA, AVAX, NEAR, SUI, DOGE, SHIB, LINK, UNI, AAVE, RNDR: 최대 50배
그 외: 최대 20배

청산·미지원 범위

증거금 소진 시 청산하며 현물이나 다른 포지션 자금을 차감하지 않습니다.
수수료·펀딩·MMR·Cross/Isolated 구분은 없습니다.
해외 거래소·USDT·지정가는 후속 범위입니다. 데이터는 기기에 저장하고 서버 계정 복구는 제공하지 않습니다.

03 계좌 준비와 광고 보상

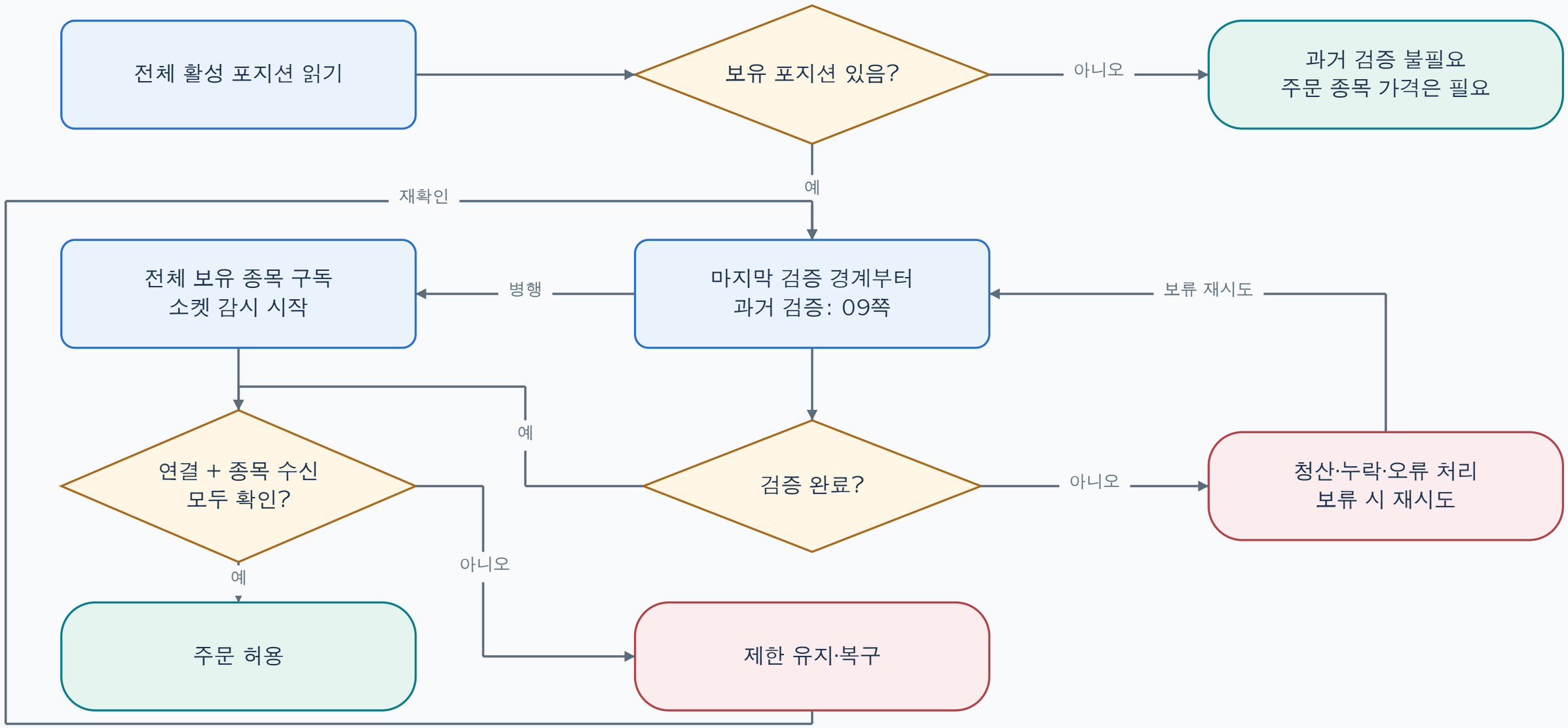
광고를 시작할 때 대상 계좌를 고정합니다. 충전은 매매 수익·PNL에 포함하지 않습니다.



SDK 중복 콜백이나 동일 보상 재처리는 보상 ID로 막습니다. 지급 실패와 광고 보상 미충족은 다른 상황입니다.

04 앱 실행·복귀: 주문 가능한 상태 만들기

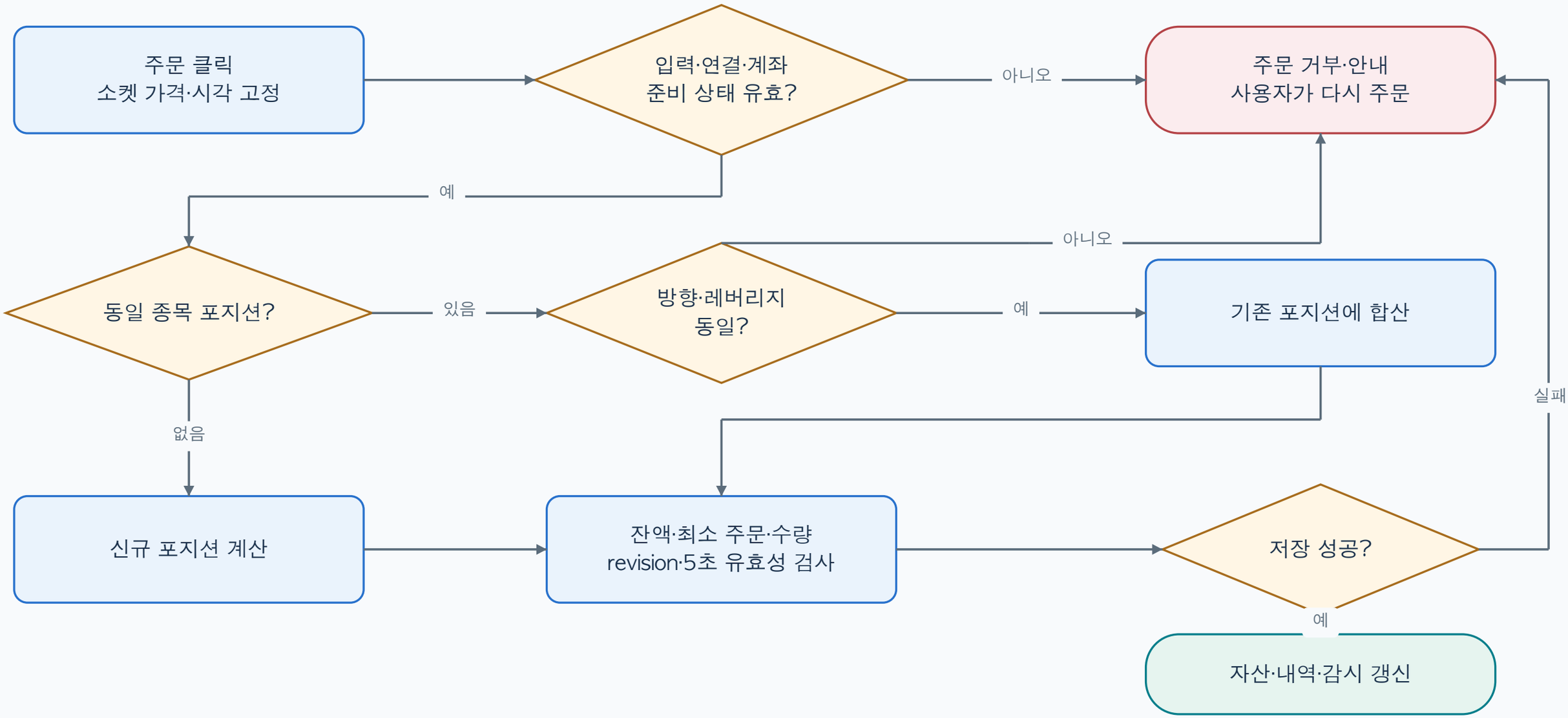
소켓 수신과 과거 검증을 연결하되, 현재가가 회복됐다는 이유로 과거 청산을 무시하지 않습니다.



준비 상태는 계좌의 활성 포지션 모두에 대해 확인합니다. 과거 청산이 확인되면 정산 후 해당 포지션을 활성 대상에서 제외합니다.

05 신규 진입·추가 진입

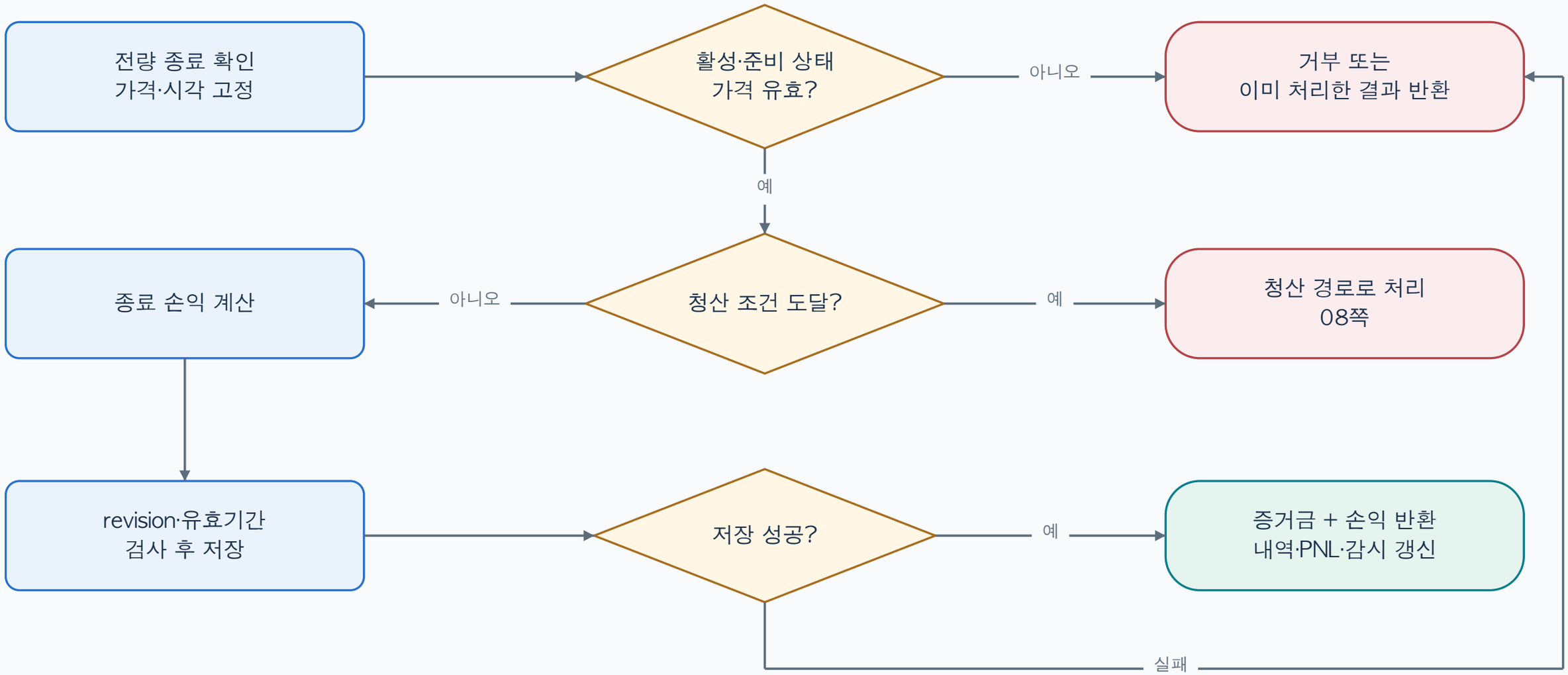
주문에는 과거 조회나 검증 완료 대기가 없습니다. 실패한 주문은 나중에 자동 실행하지 않습니다.



동일 요청 ID·동일 내용은 기존 결과를 반환합니다. ID 내용 충돌은 거부합니다. 추가 진입은 평균가·수량·증거금을 갱신하고 최초 시각·체결 이력은 보존합니다.

06 수동 전량 종료

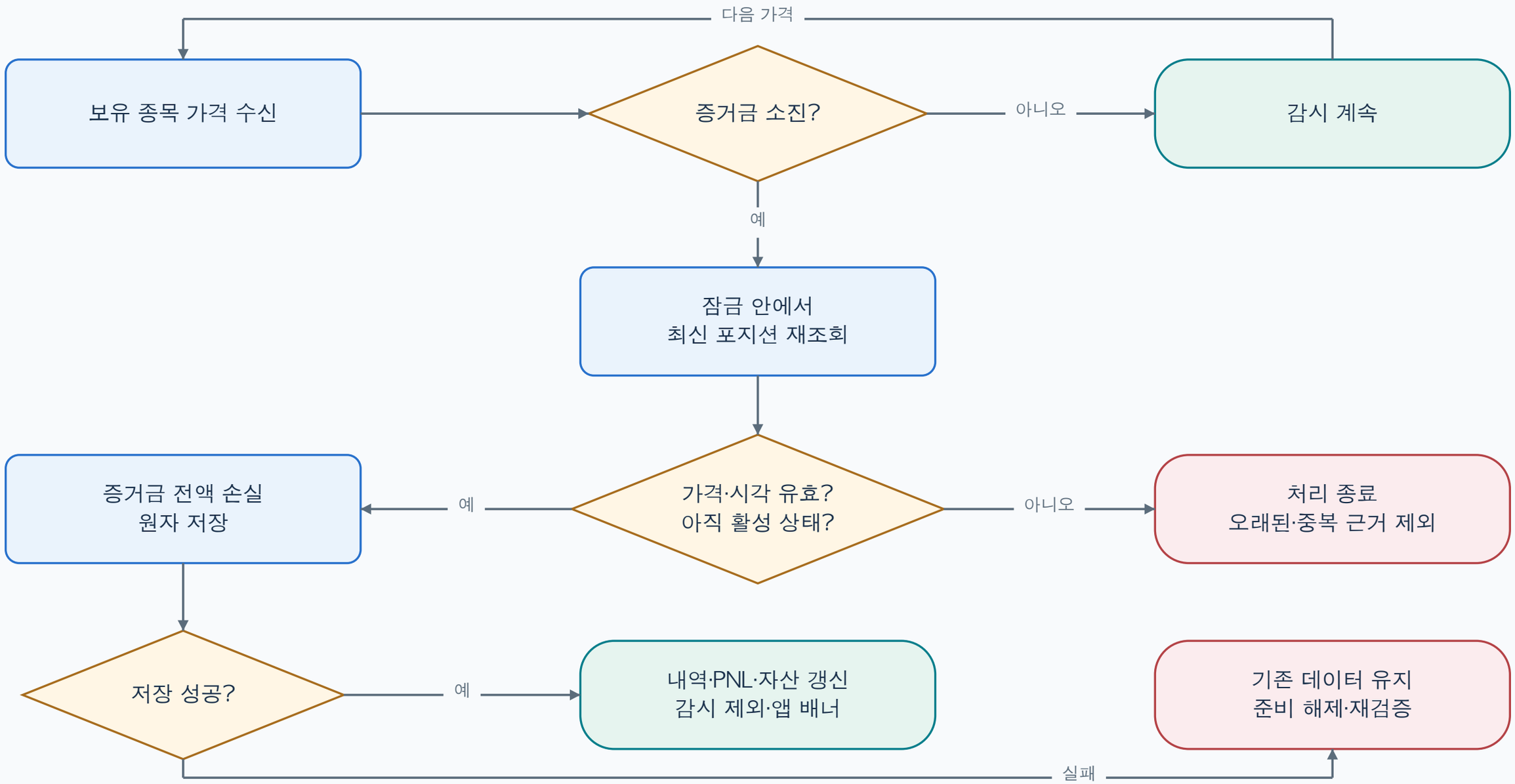
종료 확인을 누른 시점의 가격을 사용합니다. 청산 대상은 일반 종료로 처리하지 않습니다.



롱 손익 = (종료가 - 평균 진입가) × 수량. 숏은 반대입니다. 수수료·펀딩 차감은 없습니다. 저장이 끝나야 잔액 반환과 종료가 확정됩니다.

07 실시간 청산: 가격 수신 → 저장

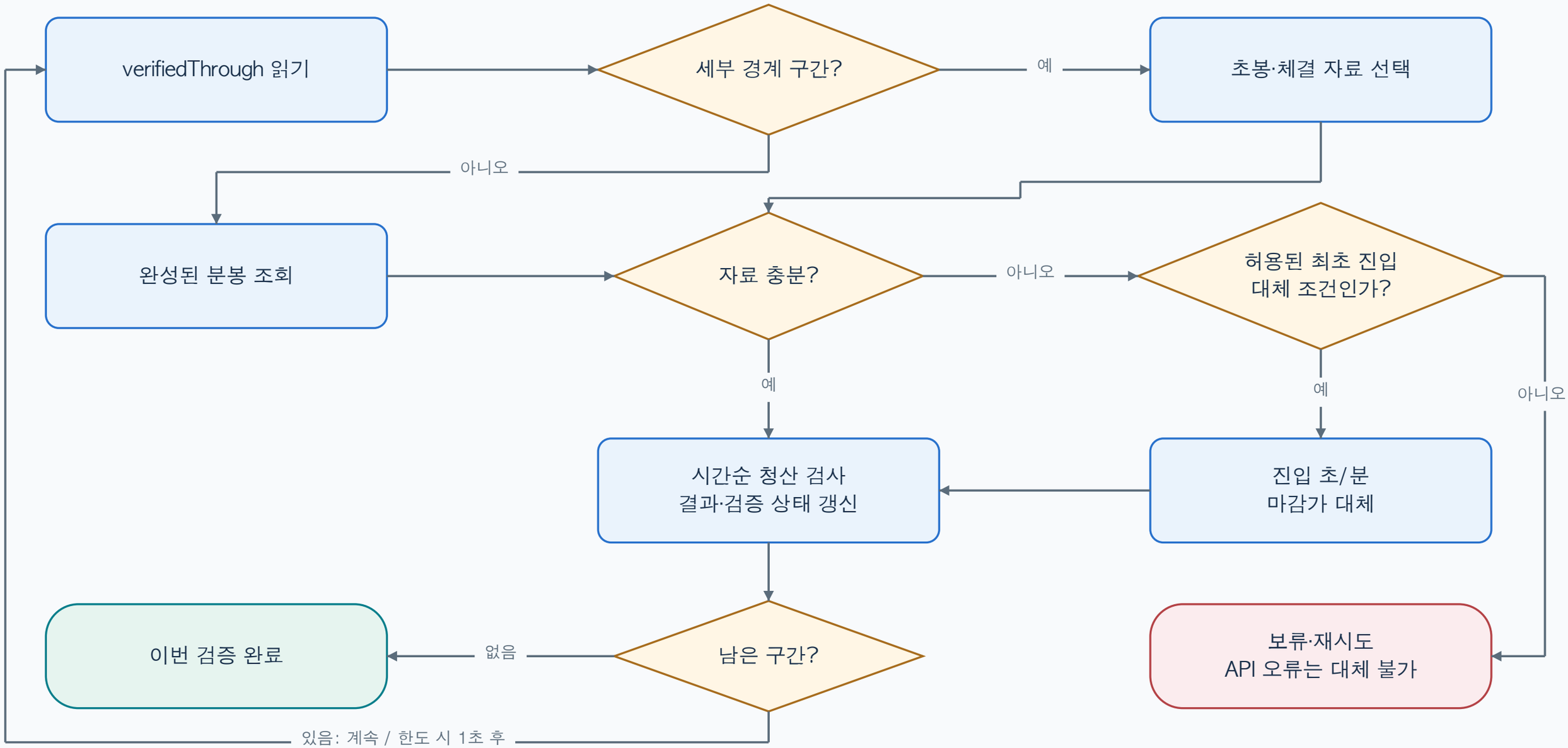
관측 가격으로 청산을 확인해도, 미조회 과거 전체를 검증 완료로 표시하지 않습니다.



예약된 증거금만 소진합니다. 현물·다른 포지션 자금을 차감하거나 가용 잔액에서 같은 손실을 두 번 빼지 않습니다.

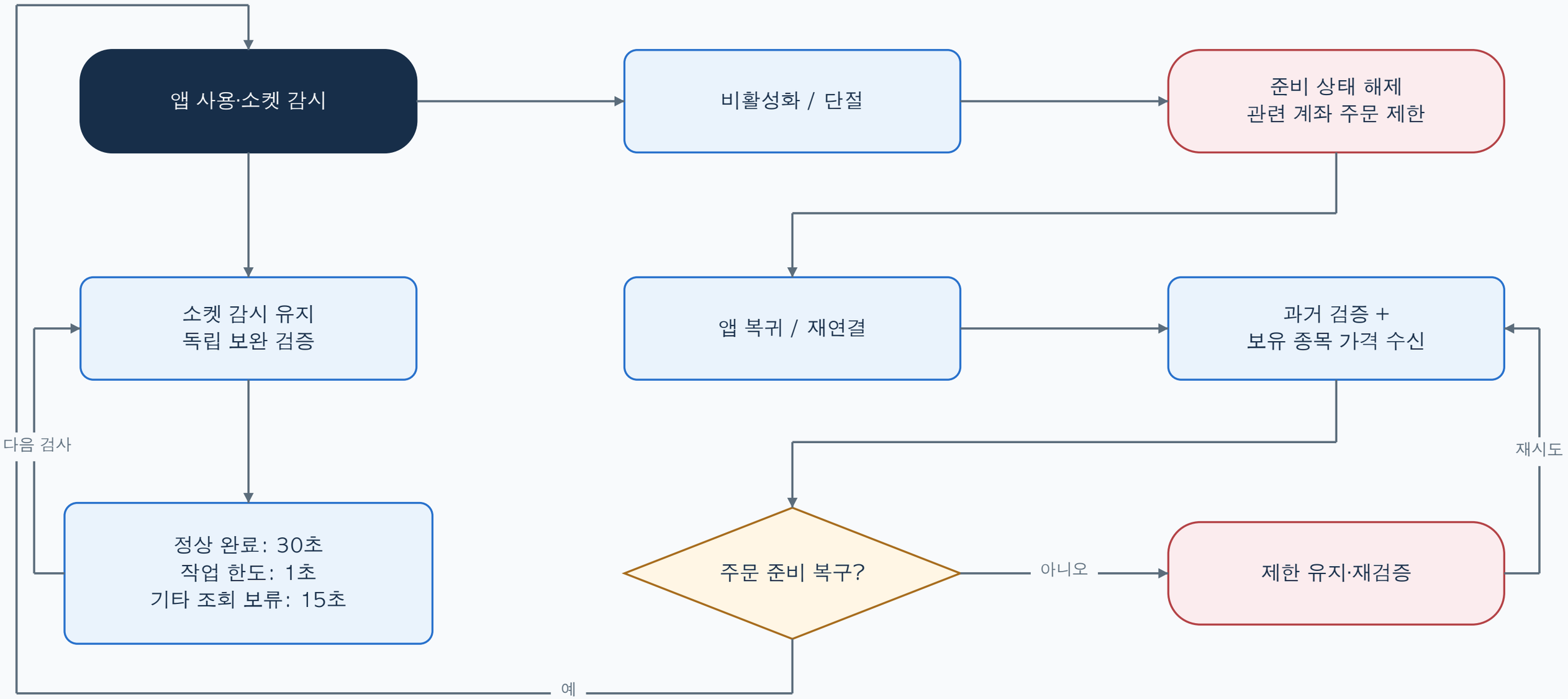
08 과거 검증: 자료 경계와 이어서 처리

롱은 저가, 숏은 고가로 검사합니다. 미검증 구간을 건너뛰지 않습니다.



09 연결 복구와 독립 보완 검증

정상 연결에서도 ticker 누락 가능성에 대비해 주문과 별도로 과거 자료를 보완합니다.



앱 비활성화 시 감시·검증을 중단합니다. 저장 실패·미지원 종목은 해당 사유만으로 주기 재시도하지 않습니다. 실패한 주문은 자동 재실행하지 않습니다.

파일 구조와 읽는 순서

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

책임의 구분

FuturesCalculator: 특정 가격에서 수량·손익·청산 여부 계산
FuturesReplayCalculator: 과거 가격 구간을 시간순으로 검사
FuturesReconciliationService: 감시·과거 검증·주문 준비 상태 관리
FuturesTradingUseCase: 진입·추가 진입·종료 요청 진행
FuturesOrderPreflight: 저장 직전 주문 유효성 확인

주문 경로

화면 → FuturesAccountReactor → FuturesTradingUseCase → FuturesReconciliationService.prepareOrder → FuturesOrderPreflight → LocalFuturesAccountRepository → FuturesCalculator → 저장 결과·화면 갱신

청산 경로

FuturesPositionSocketMonitor → FuturesReconciliationService → LocalFuturesAccountRepository → FuturesCalculator → 청산 저장·이벤트
과거 검증은 DomesticFuturesMarketRepository에서 자료를 조회하고 저장소가 FuturesReplayCalculator로 검증합니다.

탐색 기준

Domain/Model/Futures: 데이터 구조
Domain/UseCase/Futures: 계산·검증·주문 진행
Presentation/FuturesScene: 화면·Reactor·Coordinator
Data/Repository/Futures: 실제 API와 로컬 저장
파일 경로는 마지막 페이지의 목록을 참고하세요.

FuturesReconciliationService.swift

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

역할

미접속·연결 단절 중 놓친 가격 변화를 확인하고 저장된 포지션 상태를 맞추는 중심 서비스입니다. 화면에 종속되지 않으며 실시간 감시와 과거 검증을 연결합니다.

refresh / startRefresh / refresh(position:store:)

refresh()는 직렬 큐에 검증 실행을 요청합니다. startRefresh()는 이미 실행 중이면 중복 실행하지 않고 활성 포지션별 검증 결과를 모읍니다. 개별 refresh는 현재가를 확인하고, 청산 근거가 있으면 정산하며, 이어서 과거 검증 범위를 결정합니다.

process(...)

verifiedThrough부터 조회할 구간을 나누고 자료를 가져온 뒤 저장소에 검증을 요청합니다. 남은 구간이 있으면 이어가며 누락 구간을 건너뛰지 않습니다. 한 번에 처리할 페이지 수를 제한해 긴 미접속 기간을 나눠 처리합니다.

observe / processQuote / receive

observe와 processQuote는 수신 가격으로 즉시 청산 가능성을 확인합니다. 저장소가 최신 상태를 다시 확인하고 저장해야 청산이 확정됩니다. receive는 감시 소켓의 연결·가격 이벤트를 받고, 단절 시 준비 상태를 해제합니다.

prepareOrder(...)

앱 활성 여부, 고정 소켓 가격, 같은 계좌의 활성 포지션 검증 상태, 감시 연결과 종목 가격 수신을 확인합니다. 네트워크 조회나 검증 완료 대기 없이 Preflight를 반환하거나 주문을 거부합니다.

검증 서비스의 상태와 생명주기

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

주요 상태

watchedPositions: 현재 감시 중인 활성 포지션
readyPositions: 준비 완료 포지션 ID와 마지막 체결 시각
connectedExchanges: 감시 소켓 연결 거래소
receivedInstruments: 연결 후 가격을 받은 종목
watchedRevision: 감시 목록에 반영한 저장 데이터 버전

generation: 오래된 비동기 응답 차단

검증 실행을 구분하는 번호입니다. 앱 비활성화·재연결 등으로 세대가 바뀌면 이전 실행의 늦은 응답이 현재 상태를 덮어쓰지 못하게 합니다. 주문 저장 후 준비 상태를 등록할 때도 monitoringGeneration을 비교합니다.

track / suspend / publishLiquidations

track은 주문 저장 후 감시 목록과 준비 상태를 갱신합니다. suspend는 앱 비활성화 시 검증·감시를 중단하고 세션 준비 상태를 비웁니다. publishLiquidations는 저장이 끝난 청산 결과를 화면에 알려 자산 재조회와 앱 내부 배너로 연결합니다.

재실행 정책

정상 검증 후 보완 검증은 30초, 페이지 한도 도달은 1초, 자료 누락·조회 실패 등 보류는 15초 뒤 예약합니다. 저장 실패·미지원 종목은 해당 사유만으로 반복 재시도하지 않습니다. 실시간 청산은 이 주기를 기다리지 않습니다.

FuturesTradingUseCase · OrderPreflight

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

FuturesTradingUseCase.swift

사용자 매매 요청을 로컬 저장까지 진행합니다. accountSnapshot(for:)는 지원하는 선물 계좌를 확인하고 조회·생성합니다. 화면 갱신이나 캔들 분석을 직접 수행하지 않습니다.

open(...)

계좌·종목·증거금·레버리지 검사 → 요청 ID 중복 확인 → prepareOrder → 고정 가격·시각으로 주문 생성 → 저장소의 신규/추가 진입 → 감시 대상 갱신. 같은 ID와 같은 주문은 기존 결과를 반환하고, 같은 ID에 다른 내용은 충돌로 거부합니다.

close(...)

이미 완료한 요청인지 확인하고, 대상 포지션이 활성 상태인지 검사합니다. 준비 상태 확인 후 종료 주문을 만들고 저장소에서 손익·종료 이력을 저장한 뒤 감시 대상을 갱신합니다.

FuturesOrderPreflight.swift / validate(snapshot:now:)

FuturesOrderPrice는 클릭 당시 가격과 requestedAt을 담습니다. Preflight는 계좌·종목·revision·가격·검증 상태를 담은 일시적인 데이터입니다. validate는 저장 직전 revision 일치, 정상 가격·시각, 클릭 기준 5초 유효성, 검증 준비 상태와 청산 조건을 검사합니다.

시각·세대에 연결한 준비 상태

monitoredPositions는 포지션 ID와 마지막 체결 시각을 연결합니다. 추가 진입으로 조건이 바뀌었을 때 이전 준비 상태를 그대로 적용하지 않기 위해서입니다. monitoringGeneration은 준비 이후 연결·검증 세대가 바뀌었는지 확인합니다.

FuturesCalculator.swift

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

open / buildPosition

목표 포지션 금액 = 증거금 × 레버리지, 수량 = 목표 금액 ÷ 진입가입니다. 수량 단위에 맞춰 내림하여 목표 금액을 초과하지 않게 하고 최소 수량·최소 주문 금액·레버리지 상한을 검사합니다. 네트워크나 파일 저장은 하지 않습니다.

adding(...)

같은 계좌·종목·방향·레버리지만 합산합니다. 최초 주문을 유지하고 추가 체결을 additionalOrders에 보관합니다. 100원에 10개, 200원에 5개를 샀다면 평균가는 (1,000 + 1,000) ÷ 15 ≈ 133.33원입니다.

evaluate(...)

롱 손익 = 현재가 × 수량 - 총 진입 금액
숏 손익 = 총 진입 금액 - 현재가 × 수량
국내 포지션 자산 = 증거금 + 손익
ROE = 손익 ÷ 증거금 × 100
청산 = 포지션 자산 ≤ 0
표시 반올림이 아닌 실제 계산값으로 청산을 판단합니다.

종료·청산·저장 검증

estimatedLiquidationPrice: 증거금이 소진되는 예상 가격
closingQuote: 수동 종료 반환액·실현 손익
liquidationQuote: 반환액 0, 증거금 전액 손실
validate: 개별 체결 재계산과 포지션 합계 비교
storedFill: 이미 저장된 체결 복원·검증

FuturesDecimal / 국내 비용 정책

같은 파일의 FuturesDecimal은 0으로 나누기, NaN, 오버플로 등 잘못된 연산을 거부합니다. 수수료·펀딩 관련 타입이 남아 있어도 국내 경로에 적용한다는 뜻은 아니며 국내에서는 0이 아닌 값의 사용을 제한합니다.

FuturesReplayCalculator.swift

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

역할

전달받은 과거 가격 자료를 시간순으로 검사하는 계산기입니다. API 조회나 저장은 하지 않습니다. Service가 조회할 구간을 결정하고, ReplayCalculator가 그 자료의 결과를 계산합니다.

replay(...)

포지션·기존 검증 상태 확인 → 입력 시작점과 verifiedThrough 일치 확인 → 캔들 시간·개수·가격 관계와 누락 검사 → 시간순 청산 검사 → 갱신할 상태 또는 보류 사유 반환. 롱은 저가, 숏은 고가를 불리한 가격으로 사용합니다.

replayDetail(...)

초봉·체결 자료로 구성한 세부 구간을 검사합니다. 구간 사이 빈틈, 진입 이전 가격 혼입, 허용된 진입 경계 마감가 대체 규칙의 조건을 확인합니다. 불완전한 자료를 청산 없음으로 처리하지 않습니다.

validate(...)

검증 상태 자체의 정합성을 확인합니다. 검증 시각이 마지막 체결보다 이전이거나, 청산 근거 가격으로 청산 조건이 성립하지 않는 경우 등을 거부합니다. 실시간 관측 청산은 과거 미검증 구간과 observedAt을 구분하여 검사합니다.

자료가 부족한 경우

보존 기간과 경계 조건에 맞을 때만 진입 초/분 마감가 규칙을 사용합니다. 단순 네트워크 오류에는 대체 규칙을 적용하지 않습니다. 이후 정상 분봉 구간에서는 고가·저가를 계속 검사합니다.

데이터 모델 ① · Position / MarketData

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

FuturesPosition.swift

FuturesOpeningOrder: 요청 ID·계좌·종목·방향·증거금·레버리지·체결가·시각
FuturesPosition: 최초 주문·누적 수량·진입 금액·추가 체결
FuturesContractRules: 레버리지·수량 단위·최소 주문 조건
FuturesValuation: 평가손익·자산·ROE·청산 여부
FuturesSettlementQuote: 반환액·실현 손익
FuturesClosingOrder / FuturesClosedPosition: 종료 요청과 저장 결과

포지션의 주요 프로퍼티

executions: 최초 진입과 모든 추가 진입
margin: 체결별 증거금 합계
averageEntryPrice: 총 진입 금액 ÷ 총수량
lastExecutionAt: 마지막 체결 시각
leverageAtExecution: 실제 진입 레버리지
추가 진입해도 최초 진입 기록을 덮어쓰지 않습니다.

FuturesMarketData.swift

FuturesMarkQuote: 가격·timestamp
FuturesMarkCandle: OHLC와 시간 구간
FuturesTradeTick: 개별 체결
FuturesHistoryPage: 과거 조회 결과·다음 구간 정보

시세 제공 인터페이스

FuturesMarketDataProviding은 현재가·캔들 등의 제공 규약입니다.
FuturesDetailedMarketDataProviding의 boundaryHistory는 세부 경계 자료를 제공합니다. 국내 MarkQuote는 현물 기반 모의 가격을 뜻하며 해외 선물의 실제 Mark Price를 사용한다는 뜻이 아닙니다.

데이터 모델 ② · Reconciliation

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

FuturesReconciliation.swift

검증 상태·청산 기록·거래내역·계좌 평가 결과를 정의합니다. FuturesReconciliationState에는 verifiedThrough, lastVerifiedMark, liquidation, entryCloseFallbacks 등이 있습니다.

verifiedThrough와 readyPositions의 차이

verifiedThrough는 연속 검증한 구간의 끝으로, 다음 접속에서 과거 조회를 재개할 위치입니다. readyPositions는 Service의 세션 메모리에 있는 현재 주문 준비 상태입니다. 둘을 같은 체크포인트로 취급하지 않습니다.

FuturesLiquidationRecord

발생 구간, 근거 가격, 정산 시각을 보관합니다. 실시간 관측 청산의 observedAt도 구분합니다. 현재가로 청산을 관측했다고 과거 전체를 검증한 것으로 기록하지 않습니다.

transactionHistory(for:)

FuturesAccountSnapshot 확장에서 최초 진입·추가 진입·수동 종료·청산을 거래내역으로 구성합니다. 광고 지급은 매매 수익에 포함하지 않습니다.

valuation(...)

가용 잔액·예약 증거금·평가손익·실현손익·총 평가자산을 계산합니다. 이 메서드 호출만으로 잔액 저장이나 포지션 종료가 일어나지는 않습니다. 평가 표시와 확정 정산은 별개입니다.

FuturesAccountReactor와 화면 계층

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

FuturesAccountReactor.swift

화면 입력을 UseCase에 전달하고 결과를 화면 상태로 바꿉니다. mutate(action:)는 조회·진입·종료를 실행하고, execute는 진행 표시·메시지·저장 결과를 반영합니다. transform은 검증 상태·계좌 변경 알림·평가 시세 조회를 연결하고 reduce는 화면 State를 갱신합니다.

중복 요청·표시 가격

pendingOpening과 pendingClosing은 재시도 시 요청 ID를 유지합니다. 조회 실패 시 기존 유효 표시 가격을 바로 지우지 않습니다. 버튼을 누를 수 있는 상태와 실제 저장 가능 상태는 다르며 최종 판단은 Service·Preflight·저장소가 수행합니다. 주문 성공 후 과거 검증 호출은 제거됐습니다.

FuturesAccountViewController.swift

계좌·마켓 화면입니다. render로 잔액·목록을 표시하고 검색·정렬·종목 선택을 처리합니다. 화면용 시세 소켓을 제어하며 전체 포지션 감시는 별도 서비스가 담당합니다.

FuturesOrderViewController.swift

롱/숏·증거금·레버리지 입력과 종료 확인을 담당합니다. open / confirmClose에서 클릭 당시 소켓 가격을 확보해 Reactor에 전달합니다. updateEstimate는 예상값을 표시합니다. 같은 파일의 FuturesLeverageControl은 슬라이더·선택 표시·접근성 증감 동작을 담당합니다.

화면 전환·표시·연관 구현

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

FuturesInvestmentViewController.swift

보유 포지션과 투자 자산을 표시합니다. render에서 평가값·정렬을 반영하고 포지션 선택을 상세 화면으로 연결합니다.

FuturesCoordinator.swift

start로 마켓·투자내역 화면을 구성하고 open으로 상세·주문 화면, openPNL로 PNL 화면을 엽니다. 화면 생성·이동이 책임이며 거래 계산을 수행하지 않습니다.

FuturesText.swift

숫자·날짜·방향·오류 메시지 형식과 선물 화면용 UI 생성을 담당합니다. shareUnit은 거래 기록을 PNL 공유 데이터로 변환합니다.

FuturesPositionSocketMonitor / TradeReactor

SocketManager.swift 안의 FuturesPositionSocketMonitor는 watch로 거래소별 전체 보유 종목 구독을 맞추고 연결·가격 이벤트를 Service에 전달합니다. TradeReactor의 captureFuturesOrderPrice는 화면에 실제 표시된 소켓 가격과 클릭 시각을 확보합니다.

저장·조회·앱 생명주기

LocalFuturesAccountRepository: 잠금·revision·원자 저장, 주문·광고·청산의 최종 처리
DomesticFuturesMarketRepository: 국내 API 조회와 자료 변환·경계 처리
SceneDelegate: 앱 활성/비활성과 검증 시작/중단 연결, 청산 배너 표시
계산기가 청산 가능성을 계산해도 저장소가 저장해야 청산이 확정됩니다.

테스트 파일과 코드 위치

책임과 핵심 메서드 | 앞선 파일별 설명을 재배치한 기술 해설

테스트 파일

FuturesCalculatorTests.swift: 수량·손익·청산·정밀도·잘못된 값
FuturesPositionRepositoryTests.swift: 동시 저장·중복 요청·실패·광고 경합·기존 데이터 호환
FuturesReconciliationTests.swift: 과거 경계·소켓·단절·조회 제거·준비 상태·UseCase 연동
FuturesAccountReactorTests.swift: 화면 상태·가격 유지·가격 캡처·오류 표시

앱 코드 경로 · 저장소 루트는 Mobit

Mobit/Domain/UseCase/Futures/
 FuturesCalculator.swift, FuturesReplayCalculator.swift
 FuturesReconciliationService.swift, FuturesTradingUseCase.swift
 FuturesOrderPreflight.swift
Mobit/Domain/Model/Futures/
 FuturesPosition.swift, FuturesReconciliation.swift, FuturesMarketData.swift

화면·연관 코드 경로

Mobit/Presentation/FuturesScene/{Views,Reactors,Coordinator}/
Mobit/Presentation/InvestmentScene/Views/FuturesInvestmentViewController.swift
Mobit/Data/Repository/Futures/{LocalFuturesAccountRepository,DomesticFuturesMarketRepository}.swift
Mobit/Data/Network/SoketManager/SocketManager.swift
Mobit/Presentation/MainScene/Reactors/TradeReactor.swift
Mobit/Application/SceneDelegate.swift

처음 읽을 때의 순서

FuturesTradingUseCase.open → prepareOrder → FuturesOrderPreflight.validate → LocalFuturesAccountRepository.openPosition → FuturesCalculator. 한 번의 주문이 검증·계산·저장으로 이어지는 순서를 먼저 파악하면 나머지 흐름을 이해하기 쉽습니다.